CONFIDENTIAL — TLP:AMBER

PENETRATION TESTING ENGAGEMENT REPORT

TryHackMe — W1seGuy

Weak Repeating-Key XOR Encryption — Known-Plaintext Key Recovery

Platform	**TryHackMe**
Machine	**W1seGuy**
Difficulty	**Easy**
Attacker IP	**10.112.175.123**
Target IP : Port	**10.112.175.206 : 1337**
Report Date	29 September, 2026
Prepared By	MRIU-es

Table of Contents

1. Executive Summary 4
2. Objective 4
3. Scope 4
4. Methodology 4
4.1 Reconnaissance 4
4.2 Vulnerability Analysis 5
4.3 Exploitation 5
5. Findings 7
5.1 Weak Repeating-Key XOR Encryption 7
Description 7
Impact 7
Evidence 7
6. Remediation 7
7. Conclusion 8
A. Appendix — Tools Used 8

1. Executive Summary

This report documents a penetration test performed against the TryHackMe machine “W1seGuy,” an easy-difficulty challenge hosting a custom TCP service on port 1337. The service issues an XOR-encrypted value and challenges the connecting client to recover the encryption key in exchange for a flag.

Source review and traffic analysis identified a critical weakness: the encryption key is a short, five-character alphanumeric string that is reused (repeated) across the entire ciphertext. Because the plaintext format is predictable — all TryHackMe flags follow the pattern THM{...} — a known-plaintext attack recovers the key with negligible effort, fully defeating the encryption scheme.

> **RESULT**
> The encryption key was fully recovered using four bytes of guaranteed known plaintext plus a 62-value brute force on the fifth byte, and both challenge flags were retrieved. Full details are in Section 5.

2. Objective

This engagement aimed to:

- Enumerate the exposed service on the target host.
- Identify the cryptographic scheme in use and assess its strength.
- Exploit any weaknesses to recover the encryption key without prior knowledge of it.
- Retrieve the flag(s) associated with the challenge and document the process.

3. Scope

Testing was limited to the single host and port assigned by the TryHackMe platform for this room:

- Target: 10.112.175.206 : 1337
- Attacker host: 10.112.175.123
- No other hosts, ports, or services were in scope for this assessment.

4. Methodology

4.1 Reconnaissance

An initial connection was made to the target service to observe its behavior:

```
nc 10.112.175.206 1337
```

The service responded with a hex-encoded string described as an XOR-encoded flag, followed by a prompt requesting the encryption key:

```
This XOR encoded text has flag 1: <hex-encoded ciphertext>
What is the encryption key?
```

4.2 Vulnerability Analysis

Source code for the service was obtained and reviewed. Key observations:

- The encryption key is generated per connection as 5 random alphanumeric characters (random.choices over ascii_letters + digits).
- The key is applied repeatedly across the plaintext via modulo indexing (key[i % len(key)]) — a classic repeating-key XOR (Vigenère-style) cipher.
- The plaintext follows the predictable flag format THM{...}, providing four bytes of guaranteed known plaintext.
- Because XOR is reversible, any known plaintext byte directly reveals the corresponding key byte: key[i] = ciphertext[i] XOR plaintext[i].

> **ANALYST NOTE**
> Only the 4-byte prefix “THM{” can be trusted as known plaintext for any arbitrary flag. Assumptions about the 5th character (e.g. from a specific sample flag, or from flag length coincidentally being a multiple of 5) are not reliable in general and should be brute-forced instead — the 62-candidate search space is trivial to exhaust.

This combination — a short repeating key and predictable plaintext — makes the scheme vulnerable to a known-plaintext attack, fully recovering the key without brute-forcing the entire keyspace.

4.3 Exploitation

The following steps were used to recover the key and retrieve both flags:

1. Connect to the target service and capture the hex-encoded ciphertext, keeping the TCP connection open — a new connection generates a new random key.
2. Convert the hex string to raw bytes.
3. XOR the first four ciphertext bytes against the known plaintext prefix “THM{” to recover the first four key characters.
4. Brute-force the fifth key character (62 alphanumeric candidates), keeping the candidate whose decryption of the full message is printable and ends in “}”.
5. Decrypt the full ciphertext with the recovered 5-character key to obtain Flag 1.
6. Submit the recovered key on the same open connection to receive Flag 2 from the server.

Final exploit script:

```
import socket
import string

HOST, PORT = "10.112.175.206", 1337

# 1. Connect and capture the ciphertext (keep this connection open)
s = socket.create_connection((HOST, PORT))
data = b""
while b"encryption key?" not in data:
    data += s.recv(4096)

banner = data.decode()
hex_ct = banner.split("flag 1:")[1].split()[0]
ct = bytes.fromhex(hex_ct)

# 2. Recover the ONLY guaranteed known plaintext: "THM{" (4 bytes)
known_prefix = b"THM{"
key_chars = [chr(ct[i] ^ known_prefix[i]) for i in range(len(known_prefix))]

# 3. Brute-force the 5th key character (62 alphanumeric options)
flag1 = None
full_key = None
for c in string.ascii_letters + string.digits:
    candidate = key_chars + [c]
    pt = bytes(ct[i] ^ ord(candidate[i % 5]) for i in range(len(ct)))
    if pt.endswith(b"}") and all(32 <= b < 127 for b in pt):
        full_key = "".join(candidate)
        flag1 = pt.decode()
        break

print("Recovered key:", full_key)
print("Flag 1:", flag1)

# 4. Submit the key on the SAME connection to get flag 2
s.send((full_key + "\n").encode())
print("Server response:", s.recv(4096).decode())
s.close()
```

5. Findings

5.1 Weak Repeating-Key XOR Encryption

SEVERITY: HIGH

Component	Custom flag-delivery service (TCP/1337)
Weakness Type	CWE-327 / CWE-1204 — Use of a Broken or Risky Cryptographic Algorithm / Weak Key Generation
Recovered Key	**[Insert recovered key, e.g. aK3x9]**
Flag 1	**[Insert decrypted flag 1 value]**
Flag 2	**[Insert flag 2 value returned by server]**

Description

The service uses a repeating-key XOR cipher with a key length of only five characters drawn from a 62-character alphanumeric set. Combined with a predictable plaintext format, the effective attack cost drops from a 62^5 (≈916 million) brute-force search to four direct byte recoveries plus a 62-value search on the last byte — recoverable in milliseconds with no meaningful computation.

Impact

Any protection the encryption was intended to provide is defeated. An attacker with no prior knowledge of the key can recover it deterministically and obtain the protected data — in this case, the challenge flag.

Evidence

Screenshot(s) of the netcat/Python session showing the ciphertext received, the key recovered, and the server's acceptance of the key. [Insert screenshots]

6. Remediation

- Replace XOR-based encryption with a modern, authenticated algorithm such as AES-256-GCM or ChaCha20-Poly1305.
- If a one-time-pad approach is required, use a key at least as long as the message, generated per message, and never reused.
- Avoid predictable plaintext structures where feasible, or ensure the key space is large enough that known-plaintext attacks remain infeasible.
- Do not reuse or hardcode placeholder secrets (e.g. the fake flag hardcoded in setup()) on any code path that reaches production or a live challenge.

7. Conclusion

The W1seGuy challenge is a practical demonstration of why short, repeating XOR keys are unsuitable for protecting data. Using only four bytes of guaranteed known plaintext and a trivial brute force on the fifth key byte, the encryption key was fully recovered and used to retrieve both challenge flags. This engagement reinforces the importance of vetted, authenticated encryption algorithms with adequately sized, non-repeating keys.

A. Appendix — Tools Used

- netcat (nc) — manual service interaction
- Python 3 (socket, string modules) — automated exploitation script
- CyberChef — optional manual XOR decoding / verification